

Project Documentation

Introduction

- **Project Title :**Fit Track - Personalized fitness recommendations powered by AI.
- **Team Leader :** Sofia.D sofiadevan2007@gmail.com
- **Team Members :**

Karunaa.D dkarunaa18@gmail.com

Diya.A diya2006717@gmail.com

Sabana Yasmine.N sabanayasmine87@gmail.com

PROJECT OVERVIEW

Purpose:

Fit-Track is a RESTful backend service developed to simplify workout tracking and fitness management. It allows users to securely manage workout records and receive AI-based recommendations and fitness insights based on their activity data.

Core Problem Addressed:

- Difficulty maintaining workout records consistently.
- Lack of personalized fitness recommendations.
- Difficulty understanding fitness progress from raw workout data.
- Fitness information being scattered across different records or platforms.

Key Features:

- Secure user authentication using JWT and password hashing.
- CRUD operations for managing workout records.
- Search and filtering of workout history.
- AI-powered fitness recommendations using Google Gemini.
- AI-generated insights based on workout statistics.
- Input validation and centralized error handling.
- Modular backend structure for easier maintenance and scalability.

ARCHITECTURE & TECHNOLOGIES

Architecture: Model-View-Controller (MVC) based backend structure.

- **Model Layer:** Manages database schemas and fitness-related data using Mongoose.
- **Controller Layer:** Handles authentication, workout operations, and AI-related requests.
- **Route Layer:** Defines REST API endpoints and connects requests to controllers.
- **Middleware:** Handles authentication, validation, and error processing.

Runtime & Framework: Node.js with Express.js

Database: MongoDB with Mongoose

Authentication: JSON Web Token (JWT) with bcrypt password hashing

AI Engine: Google Gemini API

Testing & Development Tools: Visual Studio Code, Thunder Client / Postman, Git and GitHub

SYSTEM REQUIREMENTS

Software Requirements:

- **Node.js** – Used to run the backend server.
- **MongoDB** – Used to store user and workout data.
- **Visual Studio Code** – Used for developing and managing the project.
- **Thunder Client / Postman** – Used for testing REST API endpoints.
- **Git & GitHub** – Used for version control and project management.

Technical Requirements:

- **JavaScript** – Used for backend development.
- **Express.js** – Used to create REST API endpoints.
- **MongoDB & Mongoose** – Used for storing and managing application data.
- **JWT & bcrypt** – Used for authentication and password security.
- **Google Gemini API** – Used for AI-powered fitness recommendations and insights.
- **REST API concepts** – Used for communication between the client and backend.

FOLDER STRUCTURE

The Fit-Track backend follows a modular structure to keep the application organized and easy to maintain.

```
Fit-Track/
├── src/
│   ├── config/
│   ├── controllers/
│   ├── middleware/
│   ├── models/
│   ├── routes/
│   └── server.js
├── .env
├── package.json
└── package-lock.json
```

- **Config:** Handles database configuration.
- **Controllers:** Contains application and business logic.
- **Models:** Defines MongoDB data schemas.
- **Middleware:** Handles authentication and error processing.
- **Routes:** Defines API endpoints.
- **Server:** Starts and configures the backend application.

DATABASE DESIGN

Fit-Track uses MongoDB to store and manage user and workout information. Mongoose is used to define the database schemas and maintain the structure of the stored data.

User Schema

The User schema stores the details of registered users, including their user ID, name, email, password, and account creation date.

Workout Schema

The Workout schema stores the fitness activities recorded by users. It includes details such as workout ID, user ID, workout type, duration, calories, date, and creation time.

Relationship

Each user can have multiple workout records. The user ID is used to associate each workout with its respective user, creating a **one-to-many relationship** between the User and Workout collections.

API DOCUMENTATION

Fit-Track provides RESTful API endpoints for user authentication, workout management, search operations, and AI-based fitness services. The APIs use standard HTTP methods such as POST, GET, PUT, and DELETE.

Authentication

- POST /api/auth/register – Register a new user.
- POST /api/auth/login – Authenticate the user and generate a JWT.

Workout Management

- POST /api/workouts – Add a new workout.
- GET /api/workouts – View workout records.
- GET /api/workouts/:id – Get a specific workout by ID.
- PUT /api/workouts/:id – Update a workout.
- DELETE /api/workouts/:id – Delete a workout.

Search

- GET /api/workouts/search?q=running – Search workouts by name, category, or date.

AI Services

- POST /api/ai/recommendation – Generate AI-based fitness recommendations.
- POST /api/ai/fitness insights – Generate AI-based fitness insights.

All workout and AI endpoints are protected and require a valid JWT token.

SYSTEM WORKFLOW

The Fit-Track workflow begins with user registration and login. After successful authentication, a JWT token is generated to access protected features.

The user can then add, view, update, delete, and search workout records. The stored workout data is also used by the AI module to generate personalized fitness recommendations.

Workflow:

User → Login → JWT Authentication → Workout Management → Search → AI Recommendation → Response

TESTING & VALIDATION

Fit-Track was tested using **Thunder Client** to verify the functionality of its REST APIs.

API Testing Screenshots

User Registration

The screenshot shows a Thunder Client interface with a POST request to `http://localhost:5000/api/auth/register`. The request body is a JSON object with the following content:

```
1 {
2   "name": "Sai",
3   "email": "sai@gmail.com",
4   "password": "Sai@012"
5 }
```

The response status is **201 Created**, with a size of **276 Bytes** and a time of **1.29 s**. The response body is a JSON object with the following content:

```
1 {
2   "success": true,
3   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
4     .eyJpZCI6IjZhYjY1NzBjNTc5ZDIwNjk2MGnkNDk2ZCIsI
5     mlhdCI6MTc5MDMzNDczMiwiZXhwIjoxNzkyOTI2NzMyfQ
6     .iRbSRwIcbczfeUaHGpEDzwEsSbhB61B6_WJ00ft4R1Q",
7   "user": {
8     "id": "6ab6570c579d206960cd496d",
9     "name": "Sai",
10    "email": "sai@gmail.com"
11  }
12 }
```

User Login

The screenshot shows a Thunder Client interface with a POST request to `http://localhost:5000/api/auth/login`. The request body is a JSON object with the following content:

```
1 {
2   "email": "sai@gmail.com",
3   "password": "Sai@012"
4 }
```

The response status is **200 OK**, with a size of **276 Bytes** and a time of **2.57 s**. The response body is a JSON object with the following content:

```
1 {
2   "success": true,
3   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
4     .eyJpZCI6IjZhYjY1NzBjNTc5ZDIwNjk2MGnkNDk2ZCIsI
5     mlhdCI6MTc5MDMzNDczMiwiZXhwIjoxNzkyOTI2NzMyfQ
6     .1S9SSStawoyiDZSVhmLdBVPtwdYLZ3aDanmq9KOn4y44",
7   "user": {
8     "id": "6ab6570c579d206960cd496d",
9     "name": "Sai",
10    "email": "sai@gmail.com"
11  }
12 }
```

User Profile

GET

Query Headers² Auth¹ **Body** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

```
1
```

Status: 200 OK Size: 98 Bytes Time: 1.67 s

Response Headers⁷ Cookies Results Docs

```
1 {
2   "success": true,
3   "user": {
4     "id": "6ab63b4e373fb82e835bafda",
5     "name": "Sofia",
6     "email": "sofia@gmail.com"
7   }
8 }
```

Add Workout

POST

Query Headers² Auth¹ **Body¹** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

```
1 {
2   "workoutName": "Running",
3   "category": "Cardio",
4   "duration": 30,
5   "caloriesBurned": 250,
6   "workoutDate": "2026-09-25"
7 }
```

Status: 201 Created Size: 298 Bytes Time: 187 ms

Response Headers⁷ Cookies Results Docs

```
1 {
2   "success": true,
3   "data": {
4     "workoutName": "Running",
5     "category": "Cardio",
6     "duration": 30,
7     "caloriesBurned": 250,
8     "workoutDate": "2026-09-25T00:00:00.000Z",
9     "user": "6ab56c181388c8bdee294264",
10    "_id": "6ab7d93d5af337694d3345f2",
11    "createdAt": "2026-09-26T14:39:57.257Z",
12    "updatedAt": "2026-09-26T14:39:57.257Z",
13    "__v": 0
14  }
15 }
```

Get Workouts

GET

http://localhost:5000/api/workouts

Send

Query

Headers 2

Auth 1

Body 1

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

1

{

2

"workoutName": "running",

3

"category": "Cardio",

4

"duration": 30,

5

"caloriesBurned": 250,

6

"workoutDate": "2026-09-25"

7

}

Status: 200 OK

Size: 585 Bytes

Time: 131 ms

Response

Headers 7

Cookies

Results

Docs

</>

≡

1

{

2

"success": true,

3

"count": 2,

4

"data": [

5

{

6

"_id": "6ab7d6f95af337694d3345ef",

7

"workoutName": "running",

8

"category": "Cardio",

9

"duration": 30,

10

"caloriesBurned": 250,

11

"workoutDate": "2026-09-25T00:00:00.000Z",

12

"user": "6ab56c181388c8bdee294264",

13

"createdAt": "2026-09-26T14:30:17.960Z",

14

"updatedAt": "2026-09-26T14:30:17.960Z",

15

"__v": 0

16

},

Update Workout

PUT

http://localhost:5000/api/workouts/6ab6579b579d206960cd497a

Send

Query

Headers 2

Auth 1

Body 1

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

1

{

2

"workoutName": "Morning Jog",

3

"category": "Running",

4

"duration": 60,

5

"caloriesBurned": 650,

6

"workoutDate": "2026-06-26"

7

}

Status: 200 OK

Size: 303 Bytes

Time: 749 ms

Response

Headers 7

Cookies

Results

Docs

</>

≡

1

{

2

"success": true,

3

"data": {

4

"_id": "6ab6579b579d206960cd497a",

5

"workoutName": "Morning Jog",

6

"category": "Running",

7

"duration": 60,

8

"caloriesBurned": 650,

9

"workoutDate": "2026-06-26T00:00:00.000Z",

10

"user": "6ab63b4e373fb82e835bafda",

11

"createdAt": "2026-09-25T11:14:35.075Z",

12

"updatedAt": "2026-09-25T11:15:21.779Z",

13

"__v": 0

14

}

15

}

Delete Workout

The screenshot shows a REST client interface with a DELETE request to `http://localhost:5000/api/workouts/6ab6579b579d206960cd497a`. The response is a 200 OK status with 57 bytes and a time of 867 ms. The response body is a JSON object indicating success.

```
DELETE http://localhost:5000/api/workouts/6ab6579b579d206960cd497a
```

Query Parameters

parameter	value
-----------	-------

Response

```
1 {
2   "success": true,
3   "message": "Workout removed successfully"
4 }
```

Get Workout by ID

The screenshot shows a REST client interface with a GET request to `http://localhost:5000/api/workouts/6ab6579b579d206960cd497a`. The response is a 200 OK status with 303 bytes and a time of 420 ms. The response body is a JSON object containing workout details.

```
GET http://localhost:5000/api/workouts/6ab6579b579d206960cd497a
```

JSON Content

```
1
```

Response

```
1 {
2   "success": true,
3   "data": {
4     "_id": "6ab6579b579d206960cd497a",
5     "workoutName": "Evening Jog",
6     "category": "Running",
7     "duration": 30,
8     "caloriesBurned": 350,
9     "workoutDate": "2026-06-26T00:00:00.000Z",
10    "user": "6ab63b4e373fb82e835bafda",
11    "createdAt": "2026-09-25T11:14:35.075Z",
12    "updatedAt": "2026-09-25T11:14:35.075Z",
13    "__v": 0
14  }
15 }
```

Search Workout

Using Workout Name

The screenshot shows a REST client interface with the following details:

- Request:**
 - Method: GET
 - URL: `http://localhost:5000/api/workouts/search?q=Running`
 - Tab: Body
- Response:**
 - Status: 200 OK
 - Size: 585 Bytes
 - Time: 165 ms
 - Tab: Response
- JSON Content:** A large empty box with the number '1' in the top left corner.
- Response Body:**

```
1 {
2   "success": true,
3   "count": 2,
4   "data": [
5     {
6       "_id": "6ab7d6f95af337694d3345ef",
7       "workoutName": "running",
8       "category": "Cardio",
9       "duration": 30,
10      "caloriesBurned": 250,
11      "workoutDate": "2026-09-25T00:00:00.000Z",
12      "user": "6ab56c181388c8bdee294264",
13      "createdAt": "2026-09-26T14:30:17.960Z",
14      "updatedAt": "2026-09-26T14:30:17.960Z",
15      "__v": 0
16    },
17  ]
18 }
```

Using Workout Category

The screenshot shows a REST client interface with the following details:

- Request:**
 - Method: GET
 - URL: `http://localhost:5000/api/workouts/search?q=Cardio`
 - Tab: Body
- Response:**
 - Status: 200 OK
 - Size: 585 Bytes
 - Time: 164 ms
 - Tab: Response
- JSON Content:** A large empty box with the number '1' in the top left corner.
- Response Body:**

```
1 {
2   "success": true,
3   "count": 2,
4   "data": [
5     {
6       "_id": "6ab7d6f95af337694d3345ef",
7       "workoutName": "running",
8       "category": "Cardio",
9       "duration": 30,
10      "caloriesBurned": 250,
11      "workoutDate": "2026-09-25T00:00:00.000Z",
12      "user": "6ab56c181388c8bdee294264",
13      "createdAt": "2026-09-26T14:30:17.960Z",
14      "updatedAt": "2026-09-26T14:30:17.960Z",
15      "__v": 0
16    },
17  ]
18 }
```

Using Workout Date

The screenshot shows a REST client interface with the following details:

- Request:**
 - Method: GET
 - URL: `http://localhost:5000/api/workouts/search?q=2026-09-26`
 - Tab: Body
- Response:**
 - Status: 200 OK
 - Size: 36 Bytes
 - Time: 206 ms
 - Tab: Response
- JSON Content:** A large empty box with the number '1' in the top left corner.
- Response Body:**

```
1 {
2   "success": true,
3   "count": 0,
4   "data": []
5 }
```

AI Recommendation

AI-generated fitness recommendation.

The screenshot displays a REST client interface with a POST request to `http://localhost:5000/api/ai/workout-recommendation`. The request body is a JSON object containing user details. The response is a detailed text-based fitness recommendation.

Request:

```
POST http://localhost:5000/api/ai/workout-recommendation
```

Request Body (JSON):

```
1 {
2   "age": 22,
3   "fitnessGoal": "Weight Loss",
4   "experience": "Beginner"
5 }
```

Response:

```
1 {
2   "recommendation": "Commit to a three-day full-body resistance training schedule on non-consecutive days, such as Monday, Wednesday, and Friday. Perform three sets of eight to twelve repetitions of foundational compound movements: bodyweight squats, incline push-ups, dumbbell Romanian deadlifts, seated cable or dumbbell rows, and a thirty-second plank. Rest sixty to ninety seconds between sets, focusing entirely on controlled form rather than heavy loads.\n\nAdd thirty minutes of low-intensity steady-state cardio immediately following your lifting sessions or on your alternate days, such as Tuesday, Thursday, and Saturday. Brisk walking on a slight incline or using a stationary bicycle at a conversational pace is ideal for burning calories without causing excessive fatigue. Strive to accumulate between eight thousand and ten thousand total steps every single day to maximize your daily energy expenditure.\n\nFollow this exact regimen for the first four weeks to build movement competency and conditioning. When you can complete all three sets of twelve repetitions with ease, increase the resistance slightly or progress to standard floor\n\nnuch-line. Dain this physical routine with a
```

AI-generated fitness Insights

The screenshot displays a REST client interface with a POST request to `http://localhost:5000/api/ai/fitness-insights`. The request body is a JSON object containing workout statistics. The response is a text-based insight based on the provided data.

Request:

```
POST http://localhost:5000/api/ai/fitness-insights
```

Request Body (JSON):

```
1 {
2   "totalWorkouts": 18,
3   "averageDuration": 45,
4   "totalCaloriesBurned": 6280
5 }
```

Response:

```
1 {
2   "insight": "Logging 18 workouts with a steady 45-minute average shows exceptional consistency, yielding an impressive total burn of over 6,200 calories. Continue using 45 minutes as your ideal baseline, but begin incorporating brief high-intensity intervals or heavier resistance into the middle of your sessions to spur new adaptations. This targeted adjustment will elevate your cardiovascular output and preserve muscle without requiring any extra time in your schedule."
3 }
```

Known Issues & Limitations

- High latency during external AI completions if network encounters packet drops
- Google Gemini API rate limits restrict rapid consecutive recommendation requests
- MongoDB connection timeout if database is temporarily unavailable
- Large workout datasets may require pagination optimization for performance

FUTURE ENHANCEMENTS

Future versions of Fit-Track can be improved with the following features:

- Fitness progress charts and dashboards.
- AI-based personalized workout plans.
- Workout reminders and notifications.
- Integration with wearable fitness devices.
- More detailed fitness and activity analysis.

CONCLUSION

Fit-Track provides a secure and organized backend for managing workout records and fitness information. It combines REST APIs, MongoDB, JWT authentication, and Google Gemini AI to support workout tracking and personalized fitness insights.

The project demonstrates how backend technologies and AI can be combined to create a practical fitness management system.